1. KNN-Data Plan Recommendation

```
# ========================================
# KNN - Data Plan Recommendation
# ========================================

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# ----------------------------------------
# 1. Create Dataset
# ----------------------------------------

data = {
    'data_usage_gb': [2, 5, 8, 10, 15, 18, 20, 25, 30, 35,
                      40, 45, 50, 55, 60, 65, 70, 75, 80, 90],

    'call_minutes': [50, 80, 120, 150, 200, 250, 300, 350, 400, 450,
                     500, 550, 600, 650, 700, 750, 800, 850, 900, 950],

    'streaming_hours': [2, 4, 5, 7, 10, 12, 15, 18, 20, 22,
                        25, 28, 30, 32, 35, 38, 40, 45, 48, 50],

    'plan': ['Basic', 'Basic', 'Basic', 'Basic',
             'Standard', 'Standard', 'Standard', 'Standard',
             'Standard', 'Standard',
             'Premium', 'Premium', 'Premium', 'Premium',
             'Premium', 'Premium', 'Premium', 'Premium',
             'Premium', 'Premium']
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

# ----------------------------------------
# 2. Separate Features and Target
# ----------------------------------------

X = df[['data_usage_gb', 'call_minutes', 'streaming_hours']]
y = df['plan']

# ----------------------------------------
# 3. Split Dataset
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ----------------------------------------
# 4. Feature Scaling
# ----------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ----------------------------------------
# 5. Create KNN Model
# ----------------------------------------

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

# ----------------------------------------
# 6. Test the Model
# ----------------------------------------
```

```
y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ----------------------------------------
# 7. Recommend Plan for New Customer
# ----------------------------------------

new_customer = [[32, 450, 25]]

new_customer_scaled = scaler.transform(new_customer)

recommendation = knn.predict(new_customer_scaled)

print("\nNew Customer Usage:")
print("Data Usage       :", new_customer[0][0], "GB")
print("Call Minutes     :", new_customer[0][1])
print("Streaming Hours  :", new_customer[0][2])

print("\nRecommended Data Plan:", recommendation[0])
```

```
Dataset:
    data_usage_gb  call_minutes  streaming_hours      plan
0               2            50                2     Basic
1               5            80                4     Basic
2               8           120                5     Basic
3              10           150                7     Basic
4              15           200               10  Standard
5              18           250               12  Standard
6              20           300               15  Standard
7              25           350               18  Standard
8              30           400               20  Standard
9              35           450               22  Standard
10             40           500               25   Premium
11             45           550               28   Premium
12             50           600               30   Premium
13             55           650               32   Premium
14             60           700               35   Premium
15             65           750               38   Premium
16             70           800               40   Premium
17             75           850               45   Premium
18             80           900               48   Premium
19             90           950               50   Premium

Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

       Basic       1.00      1.00      1.00         2
     Premium       1.00      1.00      1.00         2
    Standard       1.00      1.00      1.00         2

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6


New Customer Usage:
Data Usage       : 32 GB
Call Minutes     : 450
Streaming Hours  : 25

Recommended Data Plan: Premium
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

2. Locally Weighted Regression – Electricity Consumption

```
# ========================================
# Locally Weighted Regression
# Electricity Consumption Prediction
# ========================================

import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt

# ----------------------------------------
# 1. Create Dataset
# ----------------------------------------

data = {
    'hour': [6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
             16, 17, 18, 19, 20, 21, 22, 23],

    'temperature': [24, 25, 27, 29, 31, 32, 33, 34, 35,
                    35, 34, 32, 30, 28, 27, 26, 25, 24],

    'appliance_usage': [2, 3, 4, 4, 5, 5, 6, 6, 7,
                        7, 6, 7, 8, 9, 9, 8, 6, 4],

    'consumption': [1.8, 2.1, 2.5, 2.7, 3.0, 3.2, 3.5, 3.7,
                    4.0, 4.2, 4.0, 4.5, 5.0, 5.5, 5.7, 5.2,
                    4.5, 3.2]
}

df = pd.DataFrame(data)

print("Electricity Consumption Dataset:")
print(df)

# ----------------------------------------
# 2. Prepare Features
# ----------------------------------------

X = df[['hour', 'temperature', 'appliance_usage']].values
y = df['consumption'].values

# Add bias column
X = np.c_[np.ones(X.shape[0]), X]

# ----------------------------------------
# 3. Locally Weighted Regression Function
# ----------------------------------------

def locally_weighted_regression(x_query, X, y, tau=1.0):

    # Calculate distance
    distances = np.sum((X[:, 1:] - x_query[1:]) ** 2, axis=1)

    # Calculate Gaussian weights
    weights = np.exp(-distances / (2 * tau ** 2))

    # Weight matrix
    W = np.diag(weights)

    # LWR equation:
    # theta = (X^T W X)^-1 X^T W y

    theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y

    # Prediction
    prediction = x_query @ theta

    return prediction

# ----------------------------------------
# 4. Predict Electricity Consumption
# ----------------------------------------

predictions = []

for i in range(len(X)):
    prediction = locally_weighted_regression(
        X[i], X, y, tau=5
    )

    predictions.append(prediction)

df['predicted_consumption'] = predictions
```

```
print("\nActual vs Predicted Consumption:")
print(df[['hour', 'consumption', 'predicted_consumption']])

# ----------------------------------------
# 5. Predict for a New Condition
# ----------------------------------------

# Example:
# Hour = 20
# Temperature = 28°C
# Appliance usage = 9

new_input = np.array([1, 20, 28, 9])

prediction = locally_weighted_regression(
    new_input, X, y, tau=5
)

print("\nNew Input:")
print("Hour              :", 20)
print("Temperature       :", "28°C")
print("Appliance Usage   :", 9)

print("\nPredicted Electricity Consumption:",
      round(prediction, 2), "kWh")

# ----------------------------------------
# 6. Visualization
# ----------------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
    df['hour'],
    df['consumption'],
    marker='o',
    label='Actual Consumption'
)

plt.plot(
    df['hour'],
    df['predicted_consumption'],
    marker='x',
    label='Predicted Consumption'
)

plt.xlabel("Hour of Day")
plt.ylabel("Electricity Consumption (kWh)")
plt.title("Locally Weighted Regression - Electricity Prediction")
plt.legend()
plt.grid(True)

plt.show()
```

```
Electricity Consumption Dataset:
    hour  temperature  appliance_usage  consumption
0      6           24                2          1.8
1      7           25                3          2.1
2      8           27                4          2.5
3      9           29                4          2.7
4     10           31                5          3.0
5     11           32                5          3.2
6     12           33                6          3.5
7     13           34                6          3.7
8     14           35                7          4.0
9     15           35                7          4.2
10    16           34                6          4.0
11    17           32                7          4.5
12    18           30                8          5.0
13    19           28                9          5.5
14    20           27                9          5.7
15    21           26                8          5.2
16    22           25                6          4.5
17    23           24                4          3.2

Actual vs Predicted Consumption:
    hour  consumption  predicted_consumption
0      6          1.8               1.767114
1      7          2.1               2.143749
2      8          2.5               2.549775
3      9          2.7               2.636887
4     10          3.0               3.045532
5     11          3.2               3.134574
6     12          3.5               3.568128
7     13          3.7               3.652243
8     14          4.0               4.087727
9     15          4.2               4.198920
10    16          4.0               3.916733
11    17          4.5               4.430755
12    18          5.0               4.990974
13    19          5.5               5.570495
14    20          5.7               5.650870
15    21          5.2               5.233837
16    22          4.5               4.293773
17    23          3.2               3.315137

New Input:
Hour              : 20
Temperature       : 28°C
Appliance Usage   : 9

Predicted Electricity Consumption: 5.63 kWh
```